

AI Master track. UPSaclay

PRINT AS PDF before you hit the submit button and include with your application.

louis.colbus@gmail.com [Switch account](#)



Draft saved

* Indicates required question

Email *

louis.colbus@gmail.com

First name (given name) *

Louis

Last name (family name) *

Colbus



A. Code & Repositories

- **M1 applicants:** if you lack a full project, answer using a **coursework mini-project** or Kaggle/DrivenData attempt; include a notebook link and at least one **error analysis** example.
- **M2 applicants:** include **one** deployment-oriented detail (Dockerfile snippet, FastAPI route, ONNX/TensorRT conversion, or latency measurement).

Link to one public repo (GitHub/GitLab) where **you** implemented an ML component. *

* **Required:** repo URL, file path of core model code, your username, **three** commit SHAs you authored.

* Briefly describe your role (≤ 50 words).

ATTENTION/IMPORTANT: All the code in external/DiffSinger is NOT my original work instead explicitly stated otherwise. All credit goes to the authors of the original DiffSinger paper. This was the only project I had on hand to apply.

<https://github.com/AigleSpartiate/nlp>

Username: AigleSpartiate

commits:

- db1ed498909ca2e85c2e4690f8a4ab0c118255d6
- 72f5f113ac0b0c6fa8703d4b023004602b179e12
- 0ff629aadb6f01ad349baf6ddc1cea47f856e3fe

Paste a **10–20 line** snippet from your code showing model training or data preprocessing (no boilerplate). ★

Which lines are **yours**? Why are they written that way? (≤ 60 words)

```
def clean_f0(f0, hparams):
    """Remove artifacts and smooth the raw F0 contour."""
    import scipy.signal

    # outlier removal on voiced frames
    voiced = f0 > 0
    if voiced.sum() > 2:
        mu, sigma = f0[voiced].mean(), f0[voiced].std()
        outliers = voiced & ((f0 < mu - 3 * sigma) | (f0 > mu + 3 * sigma))
        f0[outliers] = 0.0

    # median filter to suppress frame-level jitter
    kernel = hparams.get('f0_smooth_kernel', 5)
    if kernel > 1:
        f0 = scipy.signal.medfilt(f0, kernel_size=kernel)

    # interpolate short unvoiced gaps (todo: double check with and without this, might be
    bad)
    max_gap = hparams.get('f0_interp_max_gap', 4)
    nz = np.where(f0 > 0)[0]
    if len(nz) > 1:
        for i in range(len(nz) - 1):
            gap = nz[i + 1] - nz[i]
            if 1 < gap <= max_gap + 1:
                f0[nz[i]:nz[i + 1] + 1] = np.linspace(
                    f0[nz[i]], f0[nz[i + 1]], gap + 1)

    return f0
```

All of those lines are mine. They are in `data_gen_utils.py` (which otherwise was NOT written by me). I noticed unfluid transitions between notes (pitch jumps). It is hard to judge, with this change the result is barely improved at best. The interpolation might even be too much preprocessing early, worsening the possible signal the model could train on.



Show the exact command you used to run the training (e.g., `python train.py --lr ...`). *

Include the environment (conda/pip, CUDA version) and requirements.txt or environment.yml path.

`CUDA_VISIBLE_DEVICES=0 python tasks/run.py --config usr/configs/midi/e2e/opencpop/ds100_adj_rel.yaml --exp_name $MY_DS_EXP_NAME --reset`
to run training.

Environment is managed using uv. CUDA version is 12.1 for compatibility reasons.

Installing requirements.txt requires "build-essential" on Linux.

```
matplotlib
librosa==0.8.0
tqdm
pandas
numba==0.53.1
numpy==1.19.2
scipy==1.5.4
PyYAML>=5.4.1
tensorboardX
pyloudnorm
setuptools>=41.0.0
g2p_en
Resemblyzer==0.1.1.dev0
webRTCvad
tensorboard==2.6.0
scikit-learn==0.24.1
scikit-image==0.16.2
textgrid
jiwer
pycwt
PyWavelets
praat-parselmouth==0.3.3
jieba
einops
chardet
pretty-midi==0.2.9
pytorch-lightning==0.7.1
h5py==3.1.0
pypinyin==0.39.0
g2pM==0.1.2.5
protobuf==3.20.3
```



B. Data & Reproducibility

What dataset did you use **most recently**?

Name, size (#samples, #features or duration for audio), source/license, train/val/test split strategy, and one **data cleaning** step you actually performed. (≤ 80 words)

Opencpop, an open-source Chinese pop singing dataset. 3756 utterances from a single female vocalist (around 5 hours). Licensed for academic use. Includes aligned MIDI, phoneme-level durations, and lyrics. The test set consists of 5 randomly chosen songs with baseline synthesized results provided. Data cleaning step: I ran binarize.py (NOT written by me) which extracts mel-spectrograms, normalises F0 to zero mean / unit variance, and filters segments with misaligned phoneme boundaries.

Reproducibility: how did you set seeds and control nondeterminism?

List libraries where you set seeds and one source of remaining nondeterminism you **couldn't** eliminate. (≤ 60 words)

I did not try to manage nondeterminism.

C. Modeling Decisions

For one project, specify: task type (cls/reg/seq2seq/ASR/...); model family (e.g., ResNet50, XGBoost, BiLSTM, BERT-base), **why that choice over two alternatives**, and the single **most impactful hyperparameter** you tuned (value & search range). (≤ 100 words)

The goal is singing voice synthesis (seq2seq: music score $\rightarrow$ mel-spectrogram $\rightarrow$ waveform).

Model: DiffSinger, a diffusion probabilistic acoustic model with a shallow diffusion mechanism

Why over:

- (1) GAN-based SVS (e.g., GAN-Singer): DiffSinger's training is more stable
- (2) FFT-Singer: produces over-smoothed mel-spectrograms lacking detail; DiffSinger's iterative denoising preserves the most detail and I believe was the best open-source SOTA model at that time.

I used the given values for the hyperparameters.



Supervision signal: was it supervised, weakly-supervised, semi-supervised, or self-supervised?

Describe the labels or pretext task you actually used. (≤ 60 words)

It is supervised. DiffSinger's denoiser is trained with paired (music score, ground-truth mel-spectrogram) samples. The label is the Gaussian noise Epsilon added at diffusion step t ; the model predicts Epsilon_Theta and is optimized via a simplified variational bound (predicted vs actual noise).

D. Evaluation & Error Analysis

List the **primary metric** you optimized and **one trade-off** it introduced (e.g., F1 vs. latency, AUROC vs. prevalence). (≤ 60 words)

Your answer

Show **one concrete failure mode** you found via error analysis (e.g., confusion matrix cell, misclassified example with feature values, spectra snippet).

Why did it fail, and how did you attempt to fix it? (≤ 100 words)

Your answer

Attach or paste the **final validation log** line (loss/metric) and the corresponding **checkpoint filename** or SHA. Explain overfitting signs you observed (if any). (≤ 60 words)

Epoch 2: : 160001batch [11:40:19, 3.81batch/s, batch_size=41, lr=0.000125, mel=0.0801, pdur=0.00696, sdur=0.000676, step=160000, wdur=0.00385]

Checkpoint name: model_ckpt_steps_160000.ckpt (in external/DiffSinger/checkpoints/0228_opencpop_ds100_rel)



E. Compute & Systems

What hardware did you train on (CPU/GPU type, RAM/VRAM)?

Longest single run time and how you monitored it (tensorboard/wandb/custom logs). (≤ 50 words)

CPU: i7-10700K

GPU: RTX 3080 (10GB VRAM)

RAM: 32GB DDR4

The longest run time was the last one, with a total training time of 11:40:19. I initially checked that the loss was decreasing properly early on (on TensorBoard) and then simply let it run overnight.

Did you profile bottlenecks? Provide one profiler screenshot or text excerpt (e.g., PyTorch profiler) and what change you made as a result. (≤ 80 words)

I did not.

F. MLOps & Engineering Hygiene

How did you track experiments (e.g., W&B, MLflow, spreadsheets)?

Paste one **experiment ID/run URL** or table row and what decision it informed. (≤ 60 words)

I did not.



Testing: describe one **unit/integration test** you wrote for data or model code (what it checks, where it lives). (≤ 60 words)

I did not write any unit tests. The closest thing would be some validations steps that I wrote in order to ensure that the format handed to DiffSinger is correct (i.e., the size of the note groups match the word count, a DiffSinger requirement). Those can be found under the models folder in song.py or melody.py for example.

G. Teamwork & Contribution

Describe a **merge request/PR** you opened: link, title, and the reviewer's main comment you addressed. (≤ 60 words)

N/A, did everything alone.

If you worked in a team, what **part** would break without your contribution? Be specific (data loader, loss function, deployment script, etc.). (≤ 50 words)

N/A

H. Responsible & Legal AI

Cite one **dataset bias or limitation** you encountered; how did you measure or mitigate it? (≤ 80 words)

Opencpop contains a single female Mandarin singer. Therefore, the model obviously cannot generalise to male voices or other languages. When trying to run inference on English phoneme inputs the output is unintelligible (also because Chinese and English phonology differ greatly). This is an intrinsic limitation that can't be fixed without building the appropriate dataset (which are hard to find in English) and phoneme handler (which already exists in forks).



Licensing: what is the license of the code/models you used and is your repo's license compatible? (≤ 50 words)

DiffSinger's code is released under the MIT license. Opencpop is licensed for non-commercial use under the CC BY-NC-ND 4.0 license. My repo is also licensed under MIT, which is compatible. The pre-trained HifiGAN-Singing vocoder checkpoint is provided by the DiffSinger authors under the same terms.

I. Math & Understanding

For your last model, write the **exact loss function** you minimized (symbolically) and name one regularization term you used (if any). (≤ 60 words)

DiffSinger minimises a variational lower bound:

$$L = E_{\{t, M, e\}} [\|e - e_{\text{theta}}(\sqrt{a_t} * M + \sqrt{1 - a_t})\|^2]$$

where "e" is Gaussian noise, "M" is the mel-spectrogram, and "x" is the music score condition

a_t represents how much of the original data (i.e., the mel-spectrogram) remains at step t. No real explicit regularization term, the stochastic diffusion process acts itself as some

If you used cross-validation or early stopping, specify the **patience/folds** and the selection criterion. (≤ 40 words)

I did not.

Integrity Check (required) *

☒ I confirm the repositories and logs referenced above are my own work or clearly indicate collaborators. I understand that committees may check commit histories and contact supervisors.



REMEMBER TO PRINT YOUR ANSWERS AS PDF BEFORE YOU HIT THE SUBMIT BUTTON, then attach the PDF to your application. Thank you!

☒ Send me a copy of my responses.

Submit

Clear form

Never submit passwords through Google Forms.

reCAPTCHA

[Privacy](#) [Terms](#)

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms

